

Доска — документация

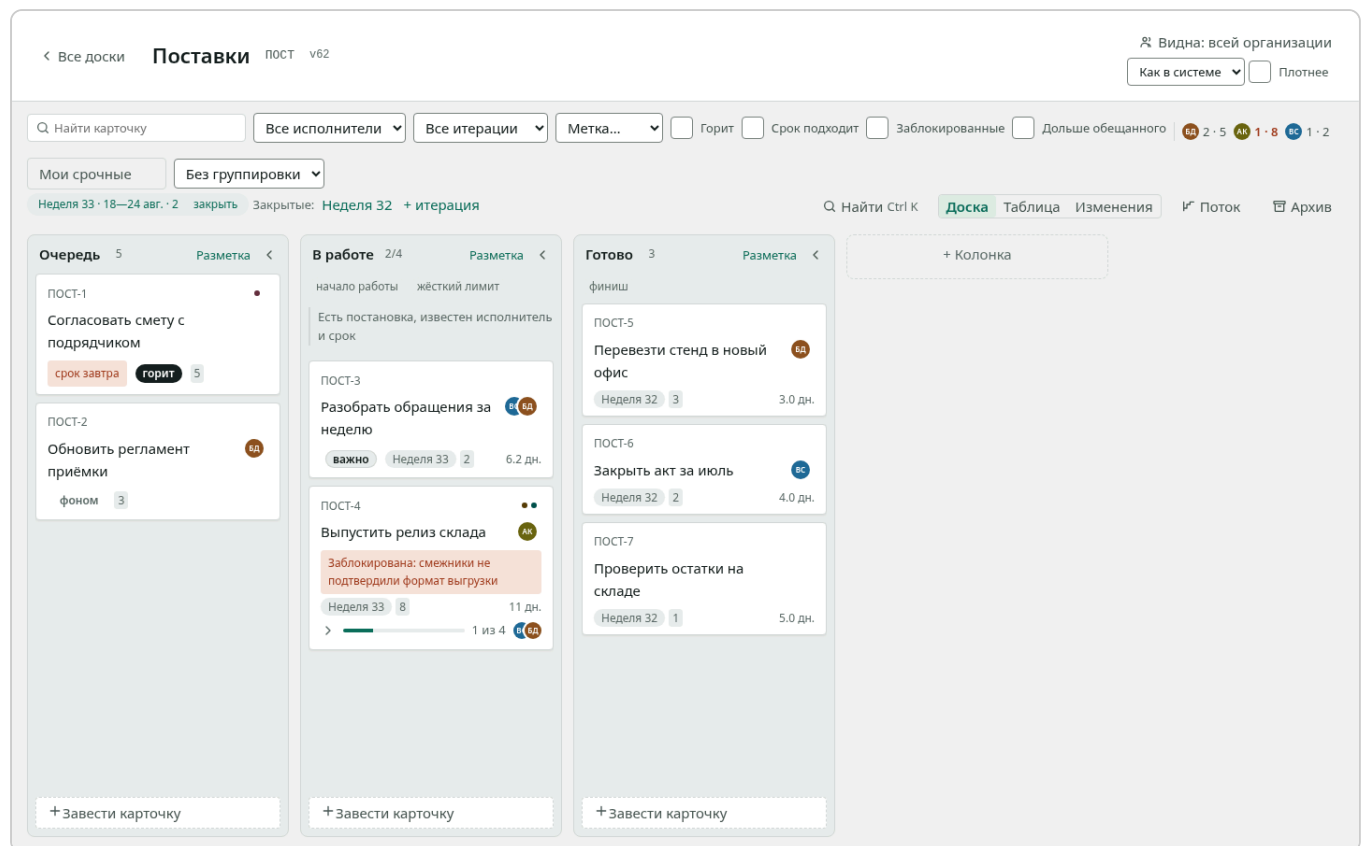
Все разделы одним файлом. Версия v0.1.0-8-gacc0df7-dirty.

Содержание

- [О продукте](#)
- [Первые 15 минут](#)
- [Как сделать](#)
- [Справочник](#)
- [Памятка](#)
- [Устройство](#)
- [Установка](#)
- [Требования](#)
- [Что менялось](#)

Доска

Канбан-трекер для команды: работа на доске, метрики потока, организация с ролями и подразделениями, интеграции. Ставится в собственный контур — облака и внешних служб не требует.



Доска

С чего начать

ВАМ НУЖНО	ОТКРОЙТЕ
Научиться вести доску	Первые пятнадцать минут
Решить конкретную задачу	Как сделать
Посмотреть списки и настройки	Справочник
Понять, почему сделано так	Устройство
Поставить и обслуживать	Установка

Разделение не случайное: обучение, инструкция, справка и объяснение отвечают на разные вопросы, и смешанные в одном тексте они мешают друг другу. Учиться удобно последовательно, задачу решать — точно, справку читать — таблицей.

Что продукт делает

Доска. Колонки-стадии, карточки-работа, ручной порядок. Перенос мышью и с клавиатуры. Лимиты колонок и правила входа. Блокировка с причиной. Подзадачи и связи между карточками — в том числе между досками разных команд.

Три вида на одни данные. Доска отвечает «как идёт работа», таблица — «где самое старое и что мы обещали», лента изменений — «что происходило».

Отбор и группировка. Восемь условий отбора, четыре разреза дорожками, сохранённые виды. Всё живёт в адресе: настроенный вид присылают ссылкой.

Метрики потока. Время цикла перцентилями, пропускная способность, работа в процессе, возраст идущего, накопление, вероятностный прогноз. Считаются из выполненной работы, а не из заполненных вручную полей.

Итерации. Спринт или релиз со сроком и отчётом по закрытии.

Организация. Роли, подразделения деревом, три вида видимости досок, приглашения по ссылке, журнал административных действий.

Интеграции. REST с описанием контракта, ключи с разрешениями, каталог SCIM 2.0, подписки на события с подписью и журналом доставок, выгрузка всех данных одним файлом.

Вход. Пароль или корпоративный провайдер (OIDC).

Чего продукт не делает

Сроков как графика и диаграмм Ганта, публичного облака с самообслуживанием, вложений в карточках. Это не недоделки: у каждого отказа записан довод, а у части — условие, при котором он будет пересмотрен. Полный список — в [требованиях](#).

Как это ставится

Один образ, одна база PostgreSQL. Разворачивается через `docker compose` на одной машине или чартом в Kubernetes; комплект для закрытого контура собирается одной командой и увозится файлом. Интернет не нужен ни при установке, ни в работе.

Подробности — в [руководстве по установке](#); что именно продукт обязуется не сломать — в [требованиях](#); что менялось между версиями — в [списке изменений](#).

Первые пятнадцать минут

Пройдите этот путь один раз — и дальше доска не будет требовать объяснений. Мы заведём доску, поставим на неё работу, проведём её до конца и позовём коллегу.

Вам понадобится открытая доска в браузере и пятнадцать минут.

Список досок

1. Заведите доску

1. На главном экране введите название в поле **«Название новой доски»**.
2. Рядом, в поле **«Ключ»**, введите короткое слово — например, `ПОСТ`. Оно станет началом номеров карточек: ПОСТ-1, ПОСТ-2. Оставьте пустым — выведем из названия.
3. Нажмите **«Завести»**.

Доска откроется сразу. В ней уже есть три колонки: «Очередь», «В работе», «Готово». Это стадии работы, и их можно менять.

2. Поставьте работу

1. Внизу колонки «Очередь» нажмите **«Завести карточку»**.
2. Напишите, что нужно сделать, и нажмите `Enter`.

Заведите три-четыре карточки: на одной доска ничего не показывает.

3. Перенесите карточку

Возьмите карточку мышью и перетащите в «В работе». Или, что быстрее: встаньте на неё с клавиатуры и нажмите `Ctrl` со стрелкой вправо.

Карточка вспыхнет на новом месте — так доска показывает, куда она уехала. Изменение сразу увидят все, у кого открыта эта доска.

4. Опишите работу подробнее

1. Щёлкните по названию карточки — откроется панель.
2. Перейдите на вкладку **«Работа»**.
3. Назначьте исполнителя, поставьте оценку и срок.

Карточка

Если работа встала — нажмите **«Заблокировать»** и напишите причину. Причина будет видна прямо на доске: «заблокирована» без объяснения не отвечает ни на один вопрос.

5. Доведите до конца

Перенесите карточку в «Готово». Через несколько таких карточек откройте **«Поток»** в шапке доски: там появятся время цикла, пропускная способность и прогноз. Метрики считаются из того, что вы уже сделали, — придумывать ничего не нужно.

6. Позовите коллегу

1. Откройте вкладку **«Команда»**.
2. В разделе **«Пригласить»** введите почту и выберите роль.
3. Нажмите **«Пригласить»** и отправьте появившуюся ссылку.

Ссылка адресная: человек с другим адресом её не примет. Показывается она один раз — в базе хранится только отпечаток.

Что дальше

- Решить конкретную задачу — [«Как сделать»](#).
- Посмотреть все возможности списком — [«Справочник»](#).
- Понять, почему доска устроена именно так, — [«Устройство»](#).

Как сделать

Ответы на конкретные вопросы. Каждый — короткий и самостоятельный: читать подряд не нужно.

Если вы здесь впервые, пройдите [первые пятнадцать минут](#) — дальше будет понятнее.

Работа на доске

Найти карточку

Нажмите **Ctrl+K** и начните печатать название. Палитра ищет и карточки, и команды доски.

Если карточка не нашлась, проверьте отбор: под отбором доска показывает не всё и говорит об этом в колонке.

Отобрать нужное

1. В строке над доской выберите условия: исполнитель, метки, «горит», «срок подходит», «заблокированные», «дольше обещанного», итерация.
2. Чтобы поделиться результатом, скопируйте адрес страницы — отбор живёт в нём.
3. Чтобы вернуться к нужному отбору позже, нажмите **«Сохранить вид»** и дайте ему имя.

Метки складываются по «и»: выбрав «срочно» и «снаружи», вы получите карточки с обеими метками.

Разложить доску дорожками

В списке **«Без группировки»** выберите, по чему раскладывать: исполнитель, метка, итерация или важность.

Дорожки — это разрез, а не другая доска: карточку между ними не перетаскать, потому что дорожка выводится из свойства карточки. Чтобы карточка сменила дорожку, смените свойство.

Сравнить карточки между собой

Нажмите **«Таблица»** в шапке доски. Это тот же набор карточек плоским списком: видно возраст, срок и оценку рядом.

Щёлкните по заголовку столбца, чтобы отсортировать. Порядок живёт в адресе — отсортированный список можно прислать.

Разбить работу на части

1. Откройте карточку, вкладка **«Работа»**.
2. В разделе подзадач нажмите **«Добавить»**.

3. Введите название части.

Часть — обычная карточка: её можно вести на другой доске, если работу делает другая команда. У задачи на доске появится полоса «сколько из скольких».

Убрать карточку с доски

Наведите на карточку, откройте меню «...» и выберите **«В архив»**. Появится сообщение с кнопкой **«Вернуть»** — на случай, если убрали не то.

Убранное лежит в разделе **«Архив»** и возвращается оттуда в любой момент. Удаление насовсем спрашивает подтверждение и не отменяется.

Вести спринт или релиз

1. В полосе над доской нажмите **«+ итерация»**.
2. Задайте название, начало и конец.
3. На карточках выберите итерацию (вкладка **«Работа»**).
4. Когда срок вышел, нажмите **«заккрыть»** — появится отчёт: что успели, что нет.

Организация

Пригласить человека

1. Вкладка **«Команда»** → раздел **«Пригласить»**.
2. Введите почту, выберите роль, нажмите **«Пригласить»**.
3. Отправьте ссылку человеку.

Ссылка действует неделю, привязана к адресу и показывается один раз. Если ссылка потерялась — отзовите приглашение и создайте новое.

Закрыть доску от посторонних

1. Откройте доску, нажмите **«Доступ»** в шапке.
2. Выберите видимость: - **«Всей организации»** — видят все; - **«Своей команде»** — видят люди подразделения; - **«Только вписанным»** — видят перечисленные поимённо.
3. Для последней добавьте людей в список.

Закрывая доску, вы вписываетесь в неё сами — иначе первым же действием потеряли бы к ней доступ.

Оформить уход сотрудника

Выберите одно из двух — они делают разное:

- **«Исключить»** — человек перестаёт быть в организации. Его карточки, комментарии и записи в журнале остаются.
- **«Удалить данные»** — имя и почта стираются, следы работы остаются без имени. Необратимо, спрашивается отдельно.

Если человек ушёл, но работа должна остаться прослеживаемой, достаточно первого.

Завести подразделения

1. Вкладка **«Структура»**.
2. Нажмите **«Завести подразделение»**, введите название.
3. Чтобы вложить одно в другое, заводите дочернее внутри узла.
4. Назначьте администратора подразделения — он будет вести своё поддерево сам.

Интеграции

Выдать ключ для интеграции

1. Вкладка «Команда» → «Ключи для интеграций».
2. Введите, для чего ключ, и выберите разрешения.
3. Нажмите «Завести» и скопируйте ключ — он показывается один раз.

Для каталога (SCIM) заведите **отдельный** ключ: такой ключ работает только со `/scim/v2` и не даёт доступа к доскам.

Получать события в свою систему

1. Вкладка «Команда» → «Подписки на события».
2. Введите название и адрес получателя, отметьте события.
3. Нажмите «Завести» и сохраните ключ подписи — им мы подписываем каждую отправку.

Если получатель уходит на обслуживание, нажмите «Приостановить»: пока пауза стоит, события не копятся, и возобновление не обернётся лавиной.

Разобраться, почему событие не дошло

1. У подписки нажмите «Доставки».
2. Посмотрите на попытки и ответ получателя.
3. Когда получателя починят, нажмите «Повторить» у недоставленного.

Повтор заодно включает подписку, если мы её отключили после долгой череды отказов.

Забрать все данные организации

Вкладка «Команда» → «Выгрузка» → «Скачать файл».

Отметьте «Добавить журнал действий», если нужен и он: журнал обычно больше всего остального вместе взятого.

Установка и обслуживание

Подробности — в [руководстве по установке](#).

Проверить, что установка сделана правильно

```
kubectl exec deploy/board -- /app/board doctor
```

Команда отвечает списком: та ли схема, тот ли адрес, достигим ли провайдер входа, доходят ли оповещения базы.

Обновить установку

```
docker load < board-image.tar.gz
helm upgrade board board-*.tgz --reuse-values --set image.tag=<версия>
kubectl exec deploy/board -- /app/board doctor
```

`--reuse-values` обязателен: без него настройки вернутся к умолчаниям чарта.

Закрыть регистрацию

Задайте `signup=closed` при установке или обновлении. Тогда организации заводит только владелец, а кнопки «Завести новую организацию» на экране входа не будет.

Справочник

Списки и таблицы: что где лежит, что как называется, что чему равно. Читается не подряд, а по нужде.

Роли в организации

РОЛЬ	ЧИТАЕТ	МЕНЯЕТ РАБОТУ	УПРАВЛЯЕТ ОРГАНИЗАЦИЕЙ
Владелец	да	да	да
Участник	да	да	нет
Наблюдатель	да	нет	нет

Сверх ролей владелец назначает:

НАЗНАЧЕНИЕ	ЧТО ДАЁТ
Администратор подразделения	ведёт своё поддерево: состав, вложенные узлы, доски
Наблюдатель области	читает всё в своём поддереве

Видимость досок

ВИДИМОСТЬ	КТО ВИДИТ
Всей организации	все люди организации
Своей команде	люди подразделения, которому принадлежит доска
Только вписанным	перечисленные поимённо

Клавиши

КЛАВИШИ	ЧТО ДЕЛАЕТ
Ctrl+K	палитра: поиск карточек и команд
Стрелки	переход между карточками
Ctrl + стрелки	перенос карточки
Enter	открыть карточку
E	переименовать, не открывая
Escape	закрыть панель, меню или палитру

Отборы

ОТБОР	ЧТО ПОКАЗЫВАЕТ
Текст	карточки, в названии которых есть подстрока
Исполнитель	чьи карточки; отдельно — «ни на ком»
Метки	карточки со всеми выбранными метками
Горит	наивысший и высокий приоритет
Срок подходит	срок сегодня, завтра, послезавтра или прошёл
Заблокированные	те, где работа встала
Дольше обещанного	те, что идут дольше обещания доски
Итерация	привязанные к итерации; отдельно — «не в итерации»

Группировки

Без группировки, по исполнителю, по метке, по итерации, по важности.

Показатели раздела «Поток»

ПОКАЗАТЕЛЬ	ЧТО ОТВЕЧАЕТ
Время цикла	сколько дней обычно занимает работа (перцентили)
Пропускная способность	сколько карточек доведено до конца по неделям
В работе	сколько работы идёт прямо сейчас
Возраст	что идёт сейчас и сколько уже идёт
Накопление	три полосы по дням: в очереди, в работе, сделано
Прогноз	сколько дней займёт довести столько-то карточек
Выброшено	сколько убрано с доски незавершёнными

Разрешения ключей интеграции

РАЗРЕШЕНИЕ	ЧТО ОТКРЫВАЕТ
<code>boards:read</code>	чтение досок и карточек
<code>boards:write</code>	изменение досок и карточек
<code>structure:read</code>	чтение структуры подразделений
<code>audit:read</code>	чтение журнала действий
<code>scim:write</code>	заведение людей и групп из каталога (только отдельным ключом)

События подписок

Карточка: создана, переименована, описание изменено, оценка изменена, приоритет изменён, срок поставлен или снят, перенесена, заблокирована, блокировка снята, связь заведена, связь снята, написано в обсуждении, своё поле заполнено, своё поле очищено, добавлена в итерацию, убрана из итерации, работа отмечена сделанной, отметка снята, убрана в архив, возвращена на доску.

Настройки установки

ПЕРЕМЕННАЯ	ЧТО ЗАДАЁТ
<code>BASE_URL</code>	адрес в браузере, без слэша на конце
<code>DATABASE_URL</code>	строка подключения к PostgreSQL
<code>LISTEN_ADDR</code>	адрес прослушивания, по умолчанию <code>:8080</code>
<code>SIGNUP</code>	кто заводит организации: <code>first</code> , <code>open</code> , <code>closed</code>
<code>WEB_DIR</code>	каталог со статикой
<code>OIDC_ISSUER</code>	адрес провайдера входа; пусто — только пароль
<code>OIDC_CLIENT_ID</code> , <code>OIDC_CLIENT_SECRET</code>	учётные данные у провайдера
<code>OIDC_ORG</code>	организация, куда попадает пришедший впервые
<code>OIDC_LABEL</code>	надпись на кнопке входа

Подкоманды

КОМАНДА	ЧТО ДЕЛАЕТ
<code>board serve</code>	работает: API, поток изменений, фоновые задачи
<code>board migrate</code>	применяет миграции и выходит
<code>board doctor</code>	проверяет, что установка сделана правильно
<code>board version</code>	отвечает, какая это версия
<code>board demo</code>	наполняет пустую базу демонстрационными данными

Пределы

что	сколько
Людей в установке	130
Одновременно открытых досок	100 получают изменение
Карточек на доске	500 без потери отклика
Строк в таблице	1000, сортировка меньше 200 мс
Первая отрисовка доски	меньше секунды

Интеграционное API

Описание контракта — на странице `/api/v1/docs` работающей установки; машиночитаемое — `/api/v1/openapi.json`. Версия стоит в адресе, причина отказа приходит кодом, повтор с тем же ключом идиempotentности безопасен.

Памятка

То, что нужно в первый день. Помещается на лист A4.

Что где

где	что там
Доски	доски организации и архив
Команда	люди, приглашения, метки, ключи, подписки, выгрузка
Структура	подразделения деревом
Шапка доски	Доска, Таблица, Изменения — три вида на одни данные; Поток — метрики; Архив — убранные карточки

Клавиши

клавиши	действие
Ctrl+K	найти карточку или команду
Стрелки	перейти между карточками
Ctrl + стрелки	перенести карточку
Enter	открыть карточку
E	переименовать, не открывая
Escape	закрыть панель, меню или палитру

Пять действий

1. **Завести доску** — на главном экране: название, ключ, «Завести».
2. **Завести карточку** — внизу колонки, Enter.

3. **Перенести** — мышью или **Ctrl** со стрелкой.
4. **Отобрать** — строка над доской; отбор живёт в адресе, вид сохраняется кнопкой **«Сохранить вид»**.
5. **Убрать** — меню **«...»** на карточке, **«В архив»**; рядом появится **«Вернуть»**.

Если что-то не так

ВИДИТЕ	ЗНАЧИТ
Полоса «Связи с сервером нет»	сервер недоступен; сделанное не потеряется
Карточка вернулась на место	перенос не дошёл; есть «Повторить»
Доска показывает не всё	включён отбор; в колонке сказано, сколько скрыто

Подробности — [«Как сделать»](#) и [«Справочник»](#).

Как это устроено и почему

Здесь не сказано, куда нажать, — сказано, почему сделано так. Читают это тогда, когда решение продукта кажется странным: почти всегда за ним стоит довод, который стоит знать.

Инструкции — в [«Как сделать»](#), списки — в [«Справочнике»](#).

Порядок карточек ручной

Карточки в колонке стоят в том порядке, в котором их поставили. Ни важность, ни срок его не меняют.

Причина: порядок отвечает на вопрос «что взять следующим», и ответ на него даёт человек, а не сортировка. Важность говорит, что важнее; срок — что обещано наружу. Это три разных вопроса, и подменять один другим значит терять два из трёх.

Лимит колонки подсвечивается, но не запрещает

Превышенный лимит виден, но перенести карточку он не мешает.

Лимит нужен, чтобы увидеть перегруз, а не чтобы спорить с человеком, который уже решил. Ни один из продуктов, где лимиты есть, не блокирует перенос: запрет ломает единственный сценарий, ради которого лимит и заводят.

Блокировка — причина, а не статус

Заблокировать карточку без объяснения нельзя: причина обязательна и видна прямо на доске.

«Заблокирована» само по себе не отвечает ни на один вопрос — ни «кого ждём», ни «что делать». Причина отвечает на оба.

Необратимое спрашивает, обратимое — нет

Убрать карточку в архив можно одним нажатием, и рядом появится «Вернуть». Удалить доску насовсем — только набрав её название.

Диалог «вы уверены?» люди закрывают не читая, поэтому подтверждение стоит там, где вернуть нельзя, и не стоит там, где можно.

Изменения расходятся сами

Открытая доска обновляется у всех, кто на неё смотрит, без перезагрузки.

Держит это одно соединение к базе на процесс, а не по соединению на вкладку: сто открытых досок — заявленный масштаб, и подписка на каждую вкладку закончилась бы исчерпанием соединений на третьем десятке.

Изоляция организаций держится базой

Данные одной организации не видны другой не потому, что так написано в коде, а потому, что так устроены политики базы.

Проверка в коде забывается при следующей правке; политика базы действует на любой запрос, включая тот, который написали в спешке. Поэтому же приложение отказывается запускаться, если роль подключения обходит политики.

Прогноз говорит о вероятности, а не о дате

«Поток» даёт три числа вместо одного: половина случаев уложится в столько-то дней, 85 % — в столько-то, 95 % — в столько-то.

Одно число там, где считается вероятность, читается как обещание. Прогноз складывает случайные недели из прошлого и отвечает ровно на один вопрос: что будет, если дальше будет как было. Если прошлого мало, он говорит и об этом.

Метрики считаются из работы, а не из отметок

Время цикла, пропускная способность и возраст берутся из того, когда карточка вошла в работу и когда её довели до конца.

Отдельные поля «начато» и «сделано», которые заполняют вручную, расходятся с реальностью в первую же занятую неделю — и расходятся молча.

Ключ доски не меняется

Ключ — начало номеров карточек — задаётся один раз.

Номер карточки попадает в переписку, в задачи смежников и в чужие системы. Сменить его значит сделать неверными все прежние ссылки разом.

Приглашение адресное и одноразовое

Ссылка привязана к почте и показывается один раз: в базе хранится только её отпечаток.

Ссылка, которую можно применить к любому адресу, — это дверь в вашу организацию, лежащая в чьей-то переписке.

Исключение и обезличивание — разные действия

Ушедший человек перестаёт быть в организации, но его карточки, комментарии и записи в журнале остаются: работа делалась, и стирать её из-за ухода нельзя. Обезличивание — отдельное действие, необратимое и спрашиваемое отдельно.

Доставка событий сдаётся вслух

Если получатель не отвечает, мы повторяем восемь раз, удваивая паузу, — около двух часов, — и после этого отключаем подписку.

Молча копить недоставленное годами хуже, чем перестать и сказать об этом: соседняя система в это время считает, что у нас ничего не происходит.

Доска

Канбан-трекер на 130 пользователей. Один процесс, одна база, два профиля развёртывания из одного образа: на обычной машине через docker compose и в Kubernetes.

Исследование рынка и обоснование архитектуры — в [research/kanban-research.html](#). Разбор устройства досок — в git-хостингах, в самом git и в чужих продуктах, с выводами для нашей схемы — в [research/boards-research.html](#), выжимка решений на одну страницу — в [research/schema-decisions.html](#).

Быстрый старт

```
make stand      # база, миграции, демонстрационные данные, сборка фронтенда
make run        # приложение на http://localhost:8099
```

Вход в стенд: `anna@example.test` / `parol12345`. Данными наполняется организация целиком — дерево подразделений, три доски всех видимостей, карточки со всеми свойствами, итерации, архив и история на три недели назад: на пустом интерфейсе не видно ни вёрстки, ни метрик.

Без данных, начисто:

```
make db         # PostgreSQL в docker на порту 55432
make web        # сборка фронтенда
make run        # приложение на http://localhost:8099
```

Или всё сразу в контейнерах:

```
make up         # http://localhost:8080
```

Первый экран предложит создать команду: на пустой установке организацию заводит тот, кто пришёл первым, и он же становится её владельцем. Дальше регистрация закрыта, и людей приглашает он. Переключается `SIGNUP`: `open` — заводить может кто угодно (так работает стенд), `closed` — никто, для установок с корпоративным входом.

После установки — проверка, что поставлено правильно:

```
docker compose run --rm app doctor      # или: /app/board doctor в поде
```

Готовность отвечает «процесс жив и база отвечает»; `doctor` отвечает на другой вопрос: та ли схема, тот ли адрес (а значит, дойдёт ли защищённая cookie), достигим ли провайдер входа, доходят ли оповещения базы, без которых доска не обновляется сама.

Документация для тех, кто пользуется: [обзор](#), [первые пятнадцать минут](#), [как сделать](#), [справочник](#), [устройство](#). Этот файл — для того, кто ставит и держит.

Что продукт обязан делать, в каких пределах и чего он делать не будет — в [REQUIREMENTS.md](#). Там же сказано, чем каждое обещание закреплено и что мы ждём от того, кто ставит.

Что уже работает

- Мультиарендность: организации, роли, приглашения по ссылке, переключение между организациями. Изоляция данных обеспечивается базой, а не кодом.
- Регистрация, вход, сессии в httpOnly cookie.
- Доски, колонки, карточки; переименование и удаление.
- Колонки размечены для потока: вид (`queue`, `in_progress`, `done`), точки старта и финиша, политика входа. Из этой разметки карточка получает `startedAt` и `finishedAt`, а из `columnEnteredAt` считается её возраст.
- WIP-лимит на колонку: по умолчанию мягкий — превышение подсвечивается, но работать не мешает. Жёсткий включается флагом и отвечает 409.
- Перетаскивание карточек мышью и с клавиатуры (`Ctrl` со стрелками на выделенной карточке), с объявлением результата для скринридера.
- Оптимистичный интерфейс: перемещение применяется мгновенно, очередь команд синхронизируется с сервером в фоне.
- Идемпотентные операции, разрешение конфликтов через 409 без перезагрузки доски, возврат карточки на место при потере связи.
- Журнал событий, из которого позже вырастет аналитика потока.

Чего ещё нет

Порядок работ и обоснование этого порядка — в [ROADMAP.md](#).

Realtime между пользователями (доска обновляется при перезагрузке), вложения, поиск, отчёты по потоку, итерации. Это этапы 1–2; что из этого необратимо задним числом — в [research/schema-decisions.html](#).

Две вещи уже есть в базе, но ещё не дошли до экрана:

- **Исход работы.** Карточка различает «сделано» и «отказались» — без этого пропускная способность считает выброшенное за сделанное.
- **Подразделения, видимость досок и наблюдение.** Вложенность команд, наследование роли вниз, наблюдение за поддеревом — всё это работает в базе и проверено тестами, но ни операций, ни экранов нет: все доски заводятся открытыми всей организации, а дерево можно построить только запросом к базе.

Устройство

<code>cmd/board/</code>	единственный исполняемый файл: <code>serve</code> и <code>migrate</code>
<code>internal/rank/</code>	дробная индексация — строковые ключи порядка карточек
<code>internal/board/</code>	доменная модель доски и применение операций
<code>internal/org/</code>	организации, состав команды, приглашения
<code>internal/auth/</code>	личности, членство, сессии
<code>internal/httpapi/</code>	маршруты, доступ, раздача фронтенда
<code>internal/store/</code>	пул соединений, области видимости, миграции
<code>migrations/</code>	SQL, встроенный в бинарник
<code>deploy/</code>	инициализация базы
<code>web/</code>	React + TypeScript

Четыре решения в схеме необратимы задним числом и заложены до того, как появились данные:

1. `cards.position` — **строковый ключ дробной индексации**. Перемещение карточки — один `UPDATE`, и два одновременных перетаскивания не ломают порядок. Алгоритм и его свойства — в `internal/rank`, с тестами.
2. `card_events` — **append-only журнал**. Cycle time, диаграмма потока и лента активности считаются из истории переходов. Восстановить её позже неоткуда.
3. `org_id` **в каждой таблице**. Сейчас бесплатно, потом — миграция всего.
4. **Точки старта и финиша на колонках**. Через них определены время цикла, возраст карточки и пропускная способность: без разметки журнал переходов копится, но не отвечает на вопрос «шла ли тогда работа». Разметить колонки можно когда угодно, ответить за прошлое — нет.

Мультиарендность

Организация — это арендатор. Личность и участие разделены:

<code>users</code>	почта и пароль, глобально уникальные — это человек
<code>memberships</code>	кто в какой организации и с какой ролью — это участие
<code>sessions</code>	помнят, в какой организации человек работает сейчас

Роль (`owner`, `member`, `viewer`) — свойство участия, а не человека: один и тот же сотрудник бывает владельцем своей команды и наблюдателем в чужой. Попасть в организацию можно только двумя способами — создать её или принять приглашение; вписать себя в чужую команду нельзя.

Изоляция обеспечивается базой, а не кодом. На всех таблицах с данными включён Row-Level Security с политикой `org_id = app_current_org()`, где `app_current_org()` читает настройку транзакции. Настройку выставляет `store.BeginTenant`, и она транзакционная — соединение возвращается в пул чистым.

Ключевое свойство: **без выставленного арендатора не видно ничего**. `current_setting` возвращает `NULL`, сравнение даёт `NULL`, строка не проходит. Забыть область — значит получить пустой ответ, а не чужие данные. Одного забытого `where org_id = ...` для утечки недостаточно.

> **Приложение не должно подключаться суперпользователем.** Для суперпользователя `>` и для роли с `BYPASSRLS` политики не действуют вообще, и вся изоляция молча `>` исчезает: запросы работают, просто отдают лишнее. Приложение проверяет это `>` при старте и отказывается запускаться. Роль для подключения создаёт `> deploy/postgres-init/10-app-role.sh`.

Приглашение — исключение из схемы: его открывают по секретной ссылке, когда организация ещё неизвестна, а у человека может не быть аккаунта. Для него заведена вторая политика, открывающая строку по хешу токена. Знание секрета и есть право — ровно то, чем приглашение является. В базе хранится только хеш, поэтому ссылке невозможно показать повторно.

Подразделения

Команда может лежать внутри команды, до пяти уровней. Предел не из осторожности: он превращает «сколько угодно предков» в массив известной длины, по которому работает индекс. Linear ограничивает пятью, GitLab допускает двадцать, но сам рекомендует не больше пяти и связывает глубину с деградацией.

Дерево хранится дважды: `teams.parent_id` — источник истины, `teams.ancestor_ids` — путь от корня до себя, пересчитываемый триггером. Так сделал GitLab, заменив рекурсивные запросы на массив с GIN-индексом, и по той же причине: рекурсивный запрос внутри политики планировщик не сплющивает, а массив ложится в индексное условие.

Роль наследуется вниз и не понижается: состоящий в направлении состоит и во всех отделах под ним. Правило «максимум из унаследованного» предсказуемо, а «где-то ниже урезали» превращает разбор доступа в раскопки. Так у всех, кто это реализовал.

Видимость досок

Внутри организации видимость перестаёт быть плоской: доска бывает открытой всем (`org`, значение по умолчанию), командной (`team`) или закрытой (`private` — только поимённо). Область транзакции поэтому шире, чем арендатор: `store.InTenant` выставляет и организацию, и человека, а `store.InOrg` — только организацию, для служебных задач, у которых нет действующего лица.

Наблюдение — строка в `observers`, а не признак у человека. Разница в том, что признак неизбежно означает «вся организация»: у GitLab наблюдатель сделан типом пользователя и поэтому видит весь инстанс, ограничить его подразделением нечем. Строка с указанием команды открывает ровно одно поддерево, без указания — всю организацию, а руководитель двух направлений выражается двумя строками. Наблюдение ортогонально роли: «видит всё» и «что меняет» — разные вопросы, иначе пришлось бы заводить `owner-observer`, `member-observer`, `viewer-observer`.

Закрытая доска — единственное исключение из «видит всё», и оно намеренное.

Журнал административных действий

На доске журналируется всё — `card_events` пишет каждое перемещение. За её пределами до недавнего времени не журналировалось ничего: кто выдал приглашение, кто поменял роль, кто перенёс отдел, кто сделал человека наблюдателем. Эти вопросы задают при разборе инцидента, и задают их всегда постфактум, поэтому `audit_events` — решение того же класса, что и сам журнал переходов: либо пишется с начала, либо не существует.

Три свойства обеспечены базой, а не обещанием:

- **Пишет триггер, а не код.** Изменение, сделанное миграцией, скриптом или руками в `psql`, попадает в журнал наравне с пришедшим через API. Журнал, который ведёт приложение, молчит ровно в тех случаях, ради которых заводился.
- **Только дописывается.** Политик `update` и `delete` нет вовсе — значит, они запрещены по умолчанию, в том числе владельцу организации.
- **Подпись нельзя подделать.** Политика вставки требует, чтобы автор записи совпадал с личностью в области транзакции.

Секреты в журнал не попадают: хеш токена приглашения вырезается, потому что политика открывает приглашение именно по хешу — знание хеша равносильно знанию ссылки.

Действие без установленной личности записывается с пустым автором, а не отвергается: журнал, роняющий обслуживание базы, снимут первым же коммитом. Читают журнал владелец и наблюдатель всей организации.

Два свойства этой схемы неочевидны, и оба обеспечены базой:

- **Закрыть доску можно только вокруг себя.** Postgres проверяет политику `select` и на новом ряде тоже, поэтому перевести доску в состояние, в котором сам её не видишь, нельзя. Иначе доска осталась бы видна одним лишь вписанным, а исправить это было бы некому: редактировать невидимую доску не может никто, включая владельца организации.
- ****Состав закрытой доски виден владельцу организации и самому участнику.**** Список участников доски найма сам по себе сведения.

Контракт операций

Клиент присылает намерение, а не вычисленное состояние:

```
POST /api/boards/{id}/operations
{ "operationId": "...", "type": "MOVE_CARD",
  "payload": { "cardId": "...", "toColumnId": "...",
               "place": "after", "afterCardId": "..." } }

→ 200 { version, patch }           применено
→ 409 { error, columnId, currentOrder, version }  конфликт
```

`place` принимает `start`, `end` или `after` (последнее — вместе с `afterCardId`) и обязателен по смыслу: умолчания «по отсутствию поля» означали бы в разных операциях разное.

`operationId` стабилен между повторами. Если ответ потерялся в сети, повтор с тем же идентификатором вернёт сохранённый результат, а не создаст вторую карточку.

Ответ `409` несёт текущий порядок затронутой колонки — клиент пересобирает её точно, без перезагрузки доски. Тем же `409` отвечает попытка положить карточку в колонку с исчерпанным жёстким лимитом.

`UPDATE_COLUMN` меняет любое подмножество свойств колонки:

```
{ "type": "UPDATE_COLUMN",
  "payload": { "columnId": "...", "kind": "in_progress",
               "isStartedPoint": true, "policy": "...",
               "wipLimit": 3, "wipLimitHard": false } }
```

Не присланное поле не меняется. Для лимита это значит, что «не трогать» и «снять» — разные намерения: снимает лимит только явный `null`.

Разработка

```
make check           # gofmt, vet, все тесты
make test            # быстрые тесты без базы
make test-integration # включая тесты против настоящего PostgreSQL
make web-dev         # Vite с горячей перезагрузкой на :5173
```

Тесты операций работают против настоящей базы: почти вся их логика живёт в SQL и транзакциях, и подменять их заглушками — значит проверять заглушки. Без `TEST_DATABASE_URL` они пропускаются.

Развёртывание

Собирается **один образ**. Compose и Helm-чарт получают его без пересборки — вся разница в переменных окружения и в том, кто запускает миграции. Второй `Dockerfile` завёл бы расхождение между профилями в тот же день.

ПЕРЕМЕННАЯ	НАЗНАЧЕНИЕ
<code>BASE_URL</code>	адрес в браузере, без слэша на конце
<code>DATABASE_URL</code>	строка подключения к PostgreSQL
<code>LISTEN_ADDR</code>	адрес прослушивания, по умолчанию <code>:8080</code>
<code>WEB_DIR</code>	каталог со статикой; пусто — не раздавать
<code>SIGNUP</code>	кто заводит организации: <code>first</code> , <code>open</code> , <code>closed</code>
<code>OIDC_ISSUER</code>	адрес провайдера входа; пусто — вход только по паролю
<code>OIDC_CLIENT_ID</code> / <code>OIDC_CLIENT_SECRET</code>	учётные данные приложения у провайдера
<code>OIDC_ORG</code>	слаг организации, куда зачисляются пришедшие впервые
<code>OIDC_LABEL</code>	надпись на кнопке входа

Миграции — отдельная команда `board migrate`, а не часть старта приложения. На одной реплике разницы нет, на двух одновременный старт разносит базу.

Роль в `DATABASE_URL` должна владеть таблицами (иначе не пройдут миграции), но **не** быть суперпользователем и не иметь `BYPASSRLS` — иначе изоляция организаций перестает действовать. Управляемые кластеры (в том числе CloudNativePG) заводят такую роль по умолчанию. Приложение проверяет это условие при старте.

За обратным прокси проверьте два условия — на них спотыкаются все:

1. `BASE_URL` совпадает с адресом в браузере. Из него клиент строит адрес WebSocket-соединения, и ошибка даёт не сообщение, а вечный спиннер.
2. Прокси пробрасывает заголовки `Upgrade` и `Connection`.

Вход через корпоративный провайдер

Настраивается переменными окружения, а не в базе. Хранить настройки по организациям было бы правильно для облачной установки, но там же пришлось бы хранить секрет клиента — то есть завести шифрование, ключ, его ротацию и вопрос «кто расшифрует, если сервер потеряли». Корпоративный вход ставят в закрытом контуре, где организация одна, а секрет уже лежит в секретах кластера. Отсюда правило: одна установка — один провайдер, никаких секретов в базе.

Обратный адрес, который нужно прописать у провайдера, собирается из `BASE_URL`: `<BASE_URL>/api/auth/oidc/callback`. Отдельной переменной для него нет намеренно — разойтись с `BASE_URL` он не должен.

Что происходит при входе:

- вернувшегося узнают по паре «издатель + sub», а не по почте: почта меняется, и человек с новой почтой иначе потерял бы свои доски;
- пришедшего впервые связывают с уже заведённой учётной записью по **подтверждённой** почте — непроверенный адрес позволил бы забрать чужую запись;
- никого не нашлось — заводят новую и зачисляют в `OIDC_ORG`. Уже состоящий хоть в одной организации никуда не добавляется: вход не должен молча менять принадлежность.

Подпись `id_token` не проверяется, и это осознанно: в код-потоке токен приходит нам напрямую с конечной точки провайдера по TLS, что спецификация прямо считает достаточным (OIDC Core, 3.1.3.7). Написанная руками проверка JWT — разбор base64, выбор ключа из JWKS, сверка `alg` — это ровно тот код, в котором ошибаются, и ошибка в нём стоит всей аутентификации. Сведения о человеке берутся с `userinfo` тем же доступом.

Восстановление на момент времени

Резервное копирование — работа базы, и чарт ею не управляет намеренно. Пример кластера с непрерывным архивированием, из которого видно, чего мы от базы ждём, — в [deploy/postgres/cloudnativepg-cluster.yaml](#).

Копировать нужно только базу. Образ воспроизводится из исходников, статика лежит внутри образа, состояния на дисках приложения нет — оно поэтому и запускается с корнем, доступным только для чтения.

Восстанавливать следует в **новый** кластер, а не поверх живого: иначе теряется возможность сравнить и вернуться, если восстановили не туда.

Что важно знать про откат базы назад по времени — это наши особенности, а не общие слова:

- **Вебхуки отправятся повторно.** Ключи повтора и отметки о доставке откатятся вместе со всем остальным, и работник разберёт исходящий ящик заново. Получатели обязаны быть готовы к повтору — в заголовке едет идентификатор доставки, по нему повтор и опознаётся.
- **Сессии откатятся тоже.** Вошедшие после точки восстановления окажутся невошедшими; выданные до неё продолжат работать. Это правильно: сессия — такие же данные, как остальные.
- **Ключи доступа, отозванные после точки, снова станут действующими.** Если откат делается из-за утечки ключа, отзывать его нужно ещё раз, уже после восстановления.
- **Открытые доски перечитаются сами.** Клиенты держат поток изменений и при обрыве переподключаются, получая снимок заново; ничего делать не нужно.

Проверять восстановление следует до аварии, а не во время: поднять копию в отдельном пространстве имён, применить чарт с `database.existingSecret`, указывающим на восстановленный кластер, и открыть доску. Проверка занимает четверть часа и отвечает на единственный вопрос, ради которого всё это делается, — «а из копии вообще поднимается?».

Установка без доступа в интернет

```
make bundle          # собирает dist/bundle-<версия>: образ, чарт, README, суммы
```

Комплект увозится файлом. На месте:

```
sha256sum -c SHA256SUMS
docker load < board-image.tar.gz          # либо skopeo copy в своё зеркало
helm install board board-*.tgz --set image.tag=<версия> \
  --set baseUrl=https://board.example.ru --set database.existingSecret=board-db
```

Контрольные суммы здесь не формальность: в закрытый контур файл едет через посредников, и «тот ли это образ» — вопрос, который зададут. По той же причине чарт позволяет ссылаться на образ по digest: тег в зеркале можно переписать, digest — нет.

Ни чарт, ни приложение никуда не ходят сами: зависимостей у чарта нет, внешних служб приложение не вызывает. Единственное исключение — корпоративный вход, и он обращается только к тому провайдеру, адрес которого указан в настройках.

Обновление в закрытом контуре

Обновлений будет больше, чем установок, а описана была только установка. Порядок тот же комплект, но короче:

```
sha256sum -c SHA256SUMS
docker load < board-image.tar.gz          # или skopeo copy в зеркало
helm upgrade board board-*.tgz --reuse-values --set image.tag=<новая версия>
kubectl exec deploy/board -- /app/board doctor
```

Что здесь важно и в каком порядке происходит:

1. **Сначала везут образ, потом обновляют выпуск.** Обновление с образом, которого нет в зеркале, оставляет поды в `ImagePullBackOff` — при `maxUnavailable: 0` старые продолжают работать, но выкладка висит.
2. **Миграции идут до замены подов**, задачей `board-migrate` с хуком `pre-upgrade`. Пока она не прошла, новые поды не поедут; упавшая задача остаётся в кластере — по ней и смотрят, почему не поехало.
3. `--reuse-values` **обязателен**, если настройки задавались при установке: без него `helm upgrade` возьмёт умолчания чарта, и `signipr`, `oidc` и всё прочее молча вернутся к исходным.
4. `doctor` **после обновления** отвечает на то, на что не отвечает готовность: та ли схема, тот ли адрес, достигим ли провайдер, доходят ли оповещения.

Что менялось между версиями — в `CHANGELOG.md`, он едет тем же комплектом. Читать его до обновления, а не после: раздел «При обновлении знать» пишется ровно для этого.

Откат. `helm rollback board` вернёт поды предыдущей версии, но **не вернёт схему базы**. Держится это на том, что миграции пишутся совместимыми с предыдущей версией приложения, и проверяется у нас (`migration_compat_test.go`). Резервная копия базы перед обновлением всё равно нужна: правило проверяем мы, а данные ваши. Если миграция была объявлена ломающей — в `CHANGELOG.md` это сказано отдельной строкой, — откат делается восстановлением базы, а не `helm rollback`.

Заведение людей из каталога (SCIM)

Провайдер выгружает сотрудников и группы по адресу `/scim/v2/...`. Ключ берётся обычный, из раздела «Ключи доступа», с единственным разрешением `scim:write`; организацию определяет сам ключ — отдельная настройка «куда заводить» была бы вторым источником правды.

Что делает наша сторона:

- **отключение снимает участие, но не удаляет человека.** Уволенный теряет доступ сразу, а его карточки, комментарии и подписи в журнале остаются подписанными им. Удалять запись значило бы переписать историю задним числом;
- **DELETE делает то же, что `active = false`.** Провайдеры расходятся: одни шлют отключение, другие удаление, третьи оба подряд;
- **группы становятся командами, а не ролями.** Роль — свойство участия в организации, и каталог о ней ничего не знает: там сотрудник состоит в отделе, а не «является владельцем». Роль назначается здесь, и выгрузка её не трогает;
- **уволенный уходит и из команд** — команда, в которой числится уволенный, вводит в заблуждение тех, кто по ней распределяет работу.

Массовые запросы (`bulk`) не поддерживаются, и `ServiceProviderConfig` об этом честно говорит: к ним прибегают на тысячах сотрудников, а установка рассчитана на сотню.

Kubernetes

Чарт лежит в `deploy/helm/board`. Зависимостей у него нет намеренно: база у заказчика своя, с резервным копированием и восстановлением на момент времени, и подсовывать ей замену внутри нашего чарта значит отвечать за чужую зону ответственности. Из этого же следует, что установка не требует доступа в интернет — нужен только образ в доступном реестре.

```
kubectl create secret generic board-db \
  --from-literal=DATABASE_URL='postgres://board:...@postgres:5432/board'

helm install board deploy/helm/board \
  --set baseUrl=https://board.example.ru \
  --set database.existingSecret=board-db \
  --set ingress.enabled=true --set ingress.host=board.example.ru
```

Пароль базы передаётся готовым секретом, а не значением чарта: из `values` он попадает и в историю выкладок, и в вывод `helm get values`, который читают шире, чем кажется.

Миграции выполняет задача с хуком `pre-install,pre-upgrade` — один раз и до того, как поедут новые поды. Упавшая задача остаётся: по ней и смотрят, почему не поехала выкладка.

Две пробы, а не одна. `/healthz` отвечает на вопрос «процесс жив» и базы не касается: если проверять живость базой, моргнувшая база перезапустит все реплики разом и к своему возвращению застанет их занятыми рестартом. `/readyz` отвечает «можно давать запросы» и базу проверяет. Он же гаснет первым при остановке: получив сигнал, реплика сначала снимает готовность, ждёт пять секунд, пока балансировщик её вычеркнет, и только потом закрывает соединения. Поэтому `terminationGracePeriodSeconds` в чарте больше, чем пауза и таймаут остановки вместе.

Требования к продукту

Что доска обязана делать, для кого, в каких пределах и чего она делать не будет.

Написано это позже всего остального, и цена запоздания измерена: три сверки подряд — интеграционное API, стратегия проверок, вход и права — нашли двадцать шесть расхождений, и почти каждое одного рода. Обещание существовало только в комментарии или в чьей-то памяти.

Требование, которое нигде не записано, нельзя ни выполнить, ни нарушить — с ним можно только разойтись.

Как это читать

Требование здесь — **то, что мы обязуемся не сломать**, а не то, что кто-то обязуется принять. Отсюда и форма: у каждого есть номер и графа «чем закреплено» — файл проверки, которая упадёт, если требование перестанет выполняться.

Незакреплённое помечено словом «не закреплено» и объяснением, почему. Пустой графы быть не должно: она означала бы, что требование держится на памяти, — то самое, ради чего документ и заведён. Проверяется это `internal/requirements`: файлы, названные в графе, обязаны существовать, номера — не повторяться, графа — не пустовать.

Чего здесь нет: как это устроено (`ROADMAP.md`, `research/*.html`), как это править (`CLAUDE.md`), какой у API контракт (`internal/httpapi/openapi.json` и страница `/api/v1/docs`).

Для кого

Команда, ведущая работу канбаном: до 130 человек в одной установке, доски в сотни карточек, несколько организаций в одной базе. Роли — владелец организации, участник, наблюдатель, администратор подразделения.

Не для: публичного облака с тысячами арендаторов, личных списков дел, управления проектами со сроками и диаграммами Ганта.

Что продукт обязан делать

№	ТРЕБОВАНИЕ	ЧЕМ ЗАКРЕПЛЕНО
T1	Доска: колонки, карточки, перенос мышью и с клавиатуры; порядок карточек ручной	<code>web/e2e/board.spec.ts</code>
T2	Перенос применяется мгновенно и переживает обрыв связи и перезагрузку страницы	<code>web/e2e/board.spec.ts</code> , <code>internal/board/regression_test.go</code>
T3	Изменение доходит до всех, у кого доска открыта, без перезагрузки	<code>internal/httpapi/stream_test.go</code> , <code>internal/httpapi/load_test.go</code>
T4	Итерации: заведение, закрытие, отчёт по закрытой	<code>internal/board/iterations_test.go</code> , <code>web/e2e/board.spec.ts</code>
T5	Метрики потока: время цикла, пропускная способность, накопление, прогноз с перцентилями	<code>internal/metrics</code> , <code>web/e2e/board.spec.ts</code>
T6	Архив карточек и досок: убранное возвращается, удалённое насовсем — нет	<code>internal/board/archive_test.go</code> , <code>web/e2e/board.spec.ts</code>
T7	Организации, приглашения по ссылке, четыре роли, подразделения деревом	<code>internal/org</code> , <code>internal/team</code> , <code>web/e2e/board.spec.ts</code>
T8	Интеграционное API с версией в адресе, кодами вместо текста и безопасным повтором	<code>internal/httpapi/contract*_test.go</code>
T9	Подписки на события с подписью, повторами и журналом доставок	<code>internal/httpapi/webhooks_test.go</code>
T10	Заведение людей из каталога по SCIM	<code>internal/httpapi/scim_test.go</code>
T11	Вход по паролю и через корпоративного провайдера (OIDC)	<code>internal/httpapi/login_test.go</code> , <code>internal/httpapi/oidc_test.go</code>
T12	Выгрузка всех данных организации одним файлом	<code>internal/httpapi/export_test.go</code>

Пределы

Числа взяты из замеров, а не назначены. За пределом продукт не обязан держать пороги отклика — но обязан работать.

№	ТРЕБОВАНИЕ	ЧЕМ ЗАКРЕПЛЕНО
П1	130 человек в установке; 100 одновременно открытых досок получают изменение	<code>internal/httpapi/load_test.go</code> (<code>make load</code>)
П2	Доска в 500 карточек открывается меньше чем за секунду и отвечает за 200 мс	<code>web/e2e/perf.spec.ts</code>
П3	Снимок доски растёт линейно с числом карточек, а не быстрее	<code>internal/board/load_test.go</code> (<code>make load</code>)
П4	Таблица на 1000 строк открывается меньше чем за секунду, сортировка — 200 мс	<code>web/e2e/perf.spec.ts</code>
П5	Тридцать человек на одной доске не медленнее очереди из тридцати	<code>internal/board/load_test.go</code> (<code>make load</code>)
П6	Соседняя доска не замедляется от занятой	<code>internal/board/load_test.go</code> (<code>make load</code>)
П7	Входной кусок клиента — не больше 400 КБ (120 КБ сжатым)	<code>web/e2e/perf.spec.ts</code>

Как продукт себя ведёт

Правила, которые важнее отдельных возможностей: их нарушение — дефект, даже если возможность работает.

№	ТРЕБОВАНИЕ	ЧЕМ ЗАКРЕПЛЕНО
П8	Организации изолированы политиками базы, а не проверками в коде	<code>internal/store/isolation_test.go</code> , <code>internal/store/identity_boundary_test.go</code>
П9	Отказ объясняет, что делать, и различается кодом, а не текстом	<code>internal/httpapi/contract_drift_test.go</code> , <code>web/src/app/forms.test.ts</code>
П10	Необратимое спрашивает, обратимое — нет; у обратимого есть отмена	<code>web/e2e/board.spec.ts</code> , <code>web/src/app/layers.test.ts</code>
П11	Всё делается с клавиатуры: перенос карточки, формы, меню	<code>web/e2e/a11y.spec.ts</code> , <code>web/src/shared/ui/Field.dom.test.tsx</code>
П12	Контраст держит WCAG 2.2 AA и порог проектирования APCA в обеих темах	<code>web/e2e/a11y.spec.ts</code>
П13	Цель нажатия не мельче 24 пикселей (WCAG 2.5.8)	<code>web/e2e/a11y.spec.ts</code>
П14	Страница не листается вбок на узком экране и не теряет содержимое при двойном размере текста	<code>web/e2e/a11y.spec.ts</code>
П15	Просьба меньше движения заменяет движение, а не выключает объяснение перемещения	<code>web/e2e/a11y.spec.ts</code> , <code>web/src/app/motion.test.ts</code>
П16	Интерфейс, отказы и журнал — по-русски	не закреплено: проверяется чтением, машинной проверки на «сказано по-русски и понятно» у нас нет

Установка и обновление

№	ТРЕБОВАНИЕ	ЧЕМ ЗАКРЕПЛЕНО
У1	Один образ на все профили развёртывания	<code>Dockerfile</code> , <code>deploy/helm/chart_test.go</code>
У2	Установка не требует доступа в интернет	<code>deploy/helm/chart_test.go</code> (у чарта нет зависимостей)
У3	Миграция совместима с предыдущей версией приложения	<code>internal/store/migration_compat_test.go</code>
У4	Цепочка миграций применяется с нуля и даёт ту же схему	<code>internal/store/migrations_test.go</code>
У5	Схема меняется только миграцией, вперёд; номера не повторяются	<code>internal/store/migration_compat_test.go</code>
У6	Настройки приложения задаются чартом, а не знанием внутренних имён	<code>deploy/helm/chart_test.go</code>
У7	После установки есть чем проверить, что она сделана правильно	<code>internal/doctor</code>
У8	У сборки есть версия, у версии — список изменений	<code>internal/version</code>
У9	Роль приложения в базе — не суперпользователь (иначе изоляция не действует)	<code>internal/store/isolation_test.go</code> , <code>internal/doctor</code>
У10	Документация едет с комплектом и открывается без сети	<code>internal/docs/render_test.go</code>

Безопасность

№	ТРЕБОВАНИЕ	ЧЕМ ЗАКРЕПЛЕНО
Б1	В зависимостях и стандартной библиотеке нет известных уязвимостей, которые мы вызываем	Makefile, цель security (govulncheck)
Б2	Код на Go не содержит находок статического разбора; исключения объявлены с причиной	Makefile, цель security (gosec)
Б3	В зависимостях клиента нет уязвимостей уровня high и выше	Makefile, цель security (npm audit)
Б4	Запрос к базе собирается параметрами; склейка допускается только со своим текстом и объявляется вслух	internal/security/sast_test.go
Б5	Клиент никогда не вставляет сырую разметку	internal/security/sast_test.go
Б6	Секретов в репозитории нет	internal/security/sast_test.go
Б7	Пароли хранятся хешем; сессия — cookie с HttpOnly и SameSite, Secure на https	internal/auth, internal/httpapi/crosssite_test.go
Б8	Межсайтовую запись останавливает хоть что-нибудь — браузер или код	internal/httpapi/crosssite_test.go
Б9	Ключ интеграции даёт только выданные разрешения и не становится владельцем	internal/httpapi/clients_test.go
Б10	Частота запросов ограничена: сбегавший ключ не мешает людям	internal/httpapi/load_test.go

Совместимость

№	ТРЕБОВАНИЕ	ЧЕМ ЗАКРЕПЛЕНО
C1	PostgreSQL 16 и новее	не закреплено: проверки идут на одной версии, матрицы версий нет
C2	Kubernetes 1.25 и новее	<code>deploy/helm/board/Chart.yaml</code> , поле <code>kubeVersion</code>
C3	Браузеры: те, где Baseline newly возрастом от года; клиент не берёт возможностей за этой границей без даты и замера	<code>web/src/app/motion.test.ts</code> , правила в <code>.claude/skills/</code> *

Чего продукт не делает

Отложено осознанно, и у каждого записано, что снимет отсрочку. Подробности — в конце `ROADMAP.md`.

- **Вложения в карточках.** Настройка `STORAGE` была и убрана: она ничего не делала, а обещала.
 - **Архивация проекта целиком.** Ждёт видимых проектов.
 - **Гранулярные разрешения** сверх четырёх ролей. Ждут реальных жалоб.
 - **Диаграмма накопления по колонкам** из журнала переходов. Считается из отметок карточки; ждёт спроса на разбивку.
 - ****Массовые запросы SCIM и настройки провайдера входа по организациям.**** Ждут масштаба, которого у установки нет.
 - **Сроки, зависимости как график, диаграмма Ганта.** Другой продукт.
 - **Публичное облако с самообслуживанием.** Регистрация закрывается настройкой ровно потому, что установка предполагается частной.
-

Чего мы ждём от того, кто ставит

Обязательства делятся, и без этого предыдущий раздел не с чем сверять.

№	ТРЕБОВАНИЕ	ЧЕМ ЗАКРЕПЛЕНО
B1	База — своя: резервные копии, восстановление на момент времени, место	не закреплено: чужая зона по определению; сказано в <code>README.md</code> и в выводе чарта
B2	Перед приложением — прокси с TLS, пробрасывающий Upgrade и Connection	не закреплено: проверяется на чужой стороне; сказано в <code>README.md</code> и <code>docker-compose.yml</code>
B3	<code>BASE_URL</code> совпадает с адресом в браузере	<code>internal/doctor</code> (проверяет вид адреса; совпадение с браузером — на стороне ставящего)
B4	Перед базой нет пула соединений в режиме транзакций	<code>internal/doctor</code> (оповещения не доходят — скажет)
B5	Резервная копия базы снимается до обновления	не закреплено: наша сторона — совместимость миграций, копия чужая; сказано в <code>README.md</code>

Что менялось

Список для тех, кто ставит и обновляет: что изменилось между двумя установками и на что смотреть после обновления. Подробности решений — в `ROADMAP.md`, он для нас; здесь — для заказчика.

Правило записи: сперва то, что меняет поведение или требует действий при обновлении, потом остальное. Версия появляется здесь раньше, чем тег, — иначе выпуск делается, а список пишется «потом».

v0.1.0 — 23 августа 2026

Первый выпуск с номером. До него версии не было вовсе: тегов в репозитории не было, комплект собирался из хеша с суффиксом `-dirty`, а в самом бинарнике версия не хранилась — ответить на «какая у нас стоит» было нечем ни заказчику, ни нам.

Установка и обновление

- `board version` — какая это версия; отвечает и тогда, когда база недоступна.
- `board doctor` — проверка того, что установка сделана правильно: применена ли схема целиком, действуют ли политики изоляции, тот ли `BASE_URL` (а значит, дойдёт ли защищённая cookie), достигим ли провайдер входа, доходят ли оповещения базы. Готовность и живость отвечают на другой вопрос — «процесс жив».
- `SIGNUP` — кто заводит организации: `first` (по умолчанию: первый пришедший становится владельцем, дальше только по приглашению), `open`, `closed`. **Смена поведения:** до этого регистрация была открыта всегда и выключить её было нечем.
- База может подниматься чартом (`postgresql.enabled=true`) — для стенда. Основное расположение прежнее: снаружи, на железе или в отдельном операторе.
- Версия установки видна на экране «Команда».

При обновлении знать

- `helm rollback` вернёт поды, но не схему базы. Миграции пишутся совместимыми с предыдущей версией приложения — на этом откат и держится, — но резервная копия перед обновлением всё равно нужна.
- Миграции идут отдельной задачей до выкатки подов; приложение не стартует на непроинициализированной базе намеренно.

Чего этот выпуск не обещает

- Установку никто ни разу не проводил: чарт проверяется отрисовкой и связностью, но не установкой в настоящий кластер.
- Вложений в карточках нет. Настройка `STORAGE` убрана: она читалась и не использовалась никем, а настройка без поведения обещает возможность и заставляет монтировать том, который никому не нужен.